VELOCIS TRANSIT NETWORK

SYSTEM DESIGN PLAN

Intelligent Campus Mobility & Smart Fleet Coordination Platform

Project Metadata

Institution	Indian Institute of Technology Roorkee
Challenge Track	Real-Time Campus Mobility and Ride Management
Development Team	Team 04
Participants	Badavath Thirupathi (23116022), Methari Sudhishna (23116062), Majji Homesh Babu (23116055), Dhanavath Mahesh (23321011)

1. Problem Understanding & Specification

The modern collegiate ecosystem at IIT Roorkee requires deterministic, low-latency infrastructure to handle last-mile intra-campus movement. Traditional transit setups often rely on fragmented, informal channels—uncoordinated phone calls, word-of-mouth booking, and manual site queuing. This causes several key operational deficits:

- Resource Sub-Optimization: Empty e-rickshaws cluster in low-demand zones while high-traffic campus loops experience critical vehicle shortages.
- Information Asymmetry: Riders face unpredictable wait times due to lack of live availability tracking, while operators have zero insight into immediate passenger demand.
- Absence of Safety & Quality Feedback Loops: No dedicated channels exist for real-time safety assistance or rider service feedback.

The Velocis Solution: This platform replaces manual coordination patterns with an event-driven web application, introducing an isolated relational datastore, full-duplex communication channels, and responsive operator/client terminals. The application handles secure role-based access control, atomic vehicle match-making, real-time map plotting, and analytical performance tracking.

2. Comprehensive System Architecture

The platform implements an Event-Driven, Decoupled 3-Tier Client-Server Architecture designed for low network latency and clean data separation.

2.1 Architectural Tier Layering Matrix

System Layer	Underlying Stack	Structural Responsibility

Presentation Layer (Frontend UI)	React 18, TypeScript, Vite, Leaflet API, Recharts Engine	Renders responsive UI, manages local reactive state hooks, handles Leaflet layers, maintains persistent WebSocket connections.
Real-Time Gateway (Pub/Sub Event Bus)	WebSocket Protocol over STOMP Frame Transport	Connects full-duplex message buses between active users, pushing immediate state dispatches.
Application Layer (Business Engine)	Java 21, Spring Boot 3, Spring Security, Hibernate ORM	Processes data streams, evaluates route tokens, isolates transaction processing, routes REST calls.
Data Persistence (Storage Tier)	PostgreSQL Database	Ensures ACID data storage, prevents concurrent entry conflicts, and logs persistent records.

2.2 Dual-Channel Client-Backend Interaction

To optimize data transport, the client browser maintains two independent, concurrent connection paths to the Spring Boot backend:

- HTTP REST Gateway (State Mutation API): Used for non-frequent, synchronous transactional data exchanges (e.g., account registration, session verification, profile updates, and submitting post-trip ratings).
- STOMP Over WebSocket Pipeline (Asynchronous Event Bus): A single, persistent full-duplex TCP socket opened immediately upon login, handling high-frequency data streams such as live GPS updates, route status shifts, and driver matching notifications—without HTTP header overhead.

3. Database Schema & Logical Structural Tables

Data persistence is handled by a relational PostgreSQL engine managed via Hibernate ORM using an auto-update schema strategy.

3.1 Core Table Definition Schemas

Table: users

Represents the core identity profile for all platform users.

- id (BIGINT, Primary Key): Unique auto-incrementing identity node.
- email (VARCHAR(255), Unique Constraint): Verified campus email asset.
- password_hash (VARCHAR(255)): Cryptographically isolated credential string.
- name (VARCHAR(100)): Full user registration name string.
- phone (VARCHAR(20)): Verified mobile contact number.
- role (ENUM): Evaluated as either PASSENGER or DRIVER.
- created_at (TIMESTAMP): System record epoch timestamp.

Table: driver_profiles

Stores operational parameters and live tracking records for certified drivers.

- id (BIGINT, Primary Key): Unique profile identifier.
- user_id (BIGINT, Foreign Key referencing users.id): Back-to-back identity link.

- availability_status (VARCHAR(30)): Live fleet state machine tracker (ONLINE, OFFLINE, BUSY).
- license_number (VARCHAR(50)): Government driving permit string.
- vehicle_number (VARCHAR(30)): Registered vehicle registration plate string.
- vehicle_type (VARCHAR(50)): Fleet classification model (e.g., E-Rickshaw).
- vehicle_capacity (INTEGER): Maximum passenger load limit.
- average_rating (DECIMAL(3,2)): Running score calculated from reviews.
- current_latitude (DECIMAL(9,6), Nullable): Live GPS horizontal coordinate node.
- current_longitude (DECIMAL(9,6), Nullable): Live GPS vertical coordinate node.

Table: rides

The central state ledger table that tracks the lifecycle of every transit transaction.

- id (BIGINT, Primary Key): Unique transaction identifier.
- passenger_id (BIGINT, Foreign Key referencing users.id): Origin client link.
- driver_id (BIGINT, Foreign Key referencing users.id, Nullable): Assigned operator link.
- pickup_location_name (VARCHAR(255)): Human-readable pickup location name.
- destination_name (VARCHAR(255)): Human-readable destination location name.
- status (VARCHAR(40)): Main lifecycle state string (REQUESTED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED).
- requested_at (TIMESTAMP): Time entry when the ride request was logged.
- started_at (TIMESTAMP, Nullable): Time entry when the operator initialized the trip.
- completed_at (TIMESTAMP, Nullable): Time entry when the operator finalized the trip destination.

Table: ratings

Stores passenger feedback and trip performance metrics.

- id (BIGINT, Primary Key): Unique rating identifier.
- ride_id (BIGINT, Foreign Key referencing rides.id, Unique Constraint): Maps exactly one review per completed trip.
- passenger_id (BIGINT, Foreign Key referencing users.id): Reviewer identity link.
- driver_id (BIGINT, Foreign Key referencing users.id): Target operator identity link.
- score (INTEGER): Numeric evaluation ranking (constrained between 1 and 5).
- feedback_text (TEXT, Nullable): Optional comment string for service audits.
- created_at (TIMESTAMP): Database log timestamp.

4. Entity Relationship Diagram (ERD) Mapping

To verify referential integrity across the datastore layer, the relational tables link together using explicit semantic cardinality paths:

- User to Driver Profile (1 : 1 Extension): Each record in users maps to exactly one record in driver_profiles via the user_id foreign key restriction.
- Passenger User to Rides (1 : N Stream): A single passenger record can initialize multiple rows inside the rides ledger table over time via the passenger_id foreign key constraint.
- Assigned Driver User to Rides (1 : N Allocation): An operator account can be linked to multiple historical ride entries via the driver_id reference column. During active queuing (status = 'REQUESTED'), this column remains NULL until a driver locks the record via the atomic endpoint.

- Completed Ride to Feedback Rating (1 : 1 Dependency): Each record in the rides matrix can match to a maximum of one review entry inside the ratings database via the ride_id foreign key, preventing duplicate ratings for a single trip.

5. System Interface Control: REST & WebSocket API Specification

5.1 RESTful Transactional API Overview Matrix

All synchronous system updates and data modifications pass through secure HTTP REST controllers.

HTTP Method	API End Route	Targeted Persona	Business Operations
POST	/api/auth/register/passenger	Visitor	Instantiates a passenger record, validates fields.
POST	/api/auth/register/driver	Visitor	Records driver vehicle parameters and licenses.
POST	/api/auth/login	Visitor	Authenticates accounts and issues a stateless JWT.
GET	/api/users/me	User	Returns account metadata and operational role flags.
POST	/api/rides	Passenger	Instantiates a transit request entry, alerts sockets.
POST	/api/rides/{id}/accept	Driver	Executes an atomic row status lock to assign a driver.
POST	/api/rides/{id}/start	Driver	Updates trip state machine to IN_PROGRESS.
POST	/api/rides/{id}/complete	Driver	Marks trip state as COMPLETED and logs end timestamp.
POST	/api/ratings	Passenger	Logs trip scores and recalculates running averages.

5.2 Structured STOMP WebSocket Messaging Subscriptions

Real-time client messaging runs on a broker configured at ws://localhost:8099/ws.

- /topic/drivers/availability (Broadcast): Pushes data frames whenever any driver toggles their online or offline state.
- /topic/rides/requests (Queue Broadcast): Instantly notifies all available online operators when a new ride request is logged.
- /topic/rides/{rideId} (Unicast Channel): Streams granular trip state transitions directly to the assigned passenger's screen.
- /topic/rides/{rideId}/driver-location (Live Stream): Streams continuous coordinate heartbeats from the driver's GPS straight to the passenger's map canvas.
- /topic/users/{userId}/notifications (Direct Target): Sends transactional alert messages directly to a specific user's interface.

6. Technical Design Decisions & Interactive UX Refactor

6.1 Bidirectional Real-Time Communication Strategy

The design uses STOMP over WebSockets instead of Server-Sent Events (SSE) or HTTP long-polling. WebSockets allow full-duplex communication over a single connection, enabling the frontend to continuously push high-frequency geolocation heartbeats from the driver's phone while simultaneously listening for inbound ride requests or passenger cancellation events without connection re-establishment overhead.

6.2 Concurrent Safe Transactional Processing

To satisfy the core requirement that a ride request can only be assigned to a single driver, the accept endpoint executes inside an isolated transaction block (@Transactional). When a driver hits the accept endpoint, the server applies a database-level lock on that specific ride row. If an overlapping request tries to lock the same row, the server flags the conflict immediately and returns an HTTP 409 Conflict state.

6.3 Secure Stateless Access Tokens

The system implements stateless JSON Web Tokens (JWT) for user sessions. Because the backend does not need to store active sessions in memory, the system can scale horizontally across multiple instances without state syncing issues. The frontend passes this token securely via standard request headers for REST endpoints and uses it as a query parameter during the initial WebSocket handshake.

6.4 Map & Open-Source Tracking Integration

Leaflet with OpenStreetMap tiles was chosen over proprietary options to eliminate developer API billing requirements. Campus pickup infrastructure markers are seeded directly into the campus_locations table. When a ride is claimed, the active coordinates of the driver's device are broadcast at regular intervals, updating the passenger's map dynamically.

6.5 Refactored Premium User Experience Controls

To build a distinct user experience, the frontend design properties were completely overhauled:

- Flipped Orientation Viewport: Moves input fields to the left side, positioning interactive branding blocks on the right to match natural visual reading patterns.
- Moving Cosmic Starfield Canvas: Uses hardware-accelerated CSS animations (space-drift) to run a slow-flowing background gradient, paired with floating e-rickshaw icons tracing pathways across the screen.
- Integrated Disconnect Control: Nests the logout action directly inside the user profile card as a unified 'Disconnect Session' control.
- Live Driver SOS Emergency Panel: Provides a permanent safety panel at the bottom of the operator's sidebar with instant click-to-call emergency shortcuts (112, Campus Medical dispatch).

7. Feature Completeness Summary

The platform successfully delivers 100% of the mandatory functional requirements specified in the challenge guidelines:

Feature	Description
Secure Role-Based Authentication	Hashed access control separated by passenger and operator flags.
Granular Driver Toggling	Full online, offline, and busy fleet state controls.
Deterministic State Machine	Trip tracking across five distinct transactional states.
Full-Duplex Messaging Bus	Real-time push updates powered by STOMP brokers.
Interactive Data Visualization	Operator analytics dashboards built with the Recharts rendering library.
Service Review System	One-time passenger rating inputs with optional text feedback.
Geospatial Mapping Integration	OpenStreetMap rendering tiles with automated marker bounds checking.
Live Driver GPS Tracking	High-frequency location streaming from driver devices straight to passenger maps.